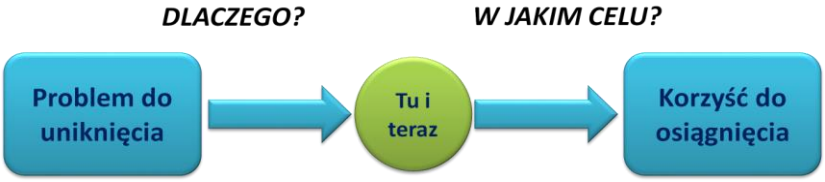
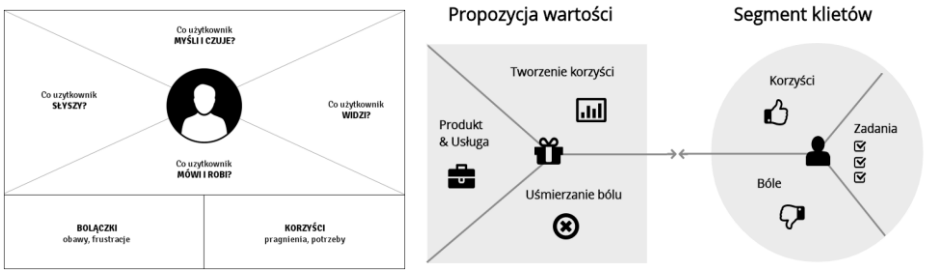
Układ pracy inżynierskiej - programistycznej

1. **Strona tytułowa** *(zgodna ze wzorem).*
2. **Streszczenie** *(na ok. pół strony zawierające informacje o pracy). Poniżej dodatkowo informacje o podziale prac w przypadku projektów zespołowych oraz wykorzystanie AI.*
3. **Spis treści** (automatyczny, oparty na stylach).
4. **Geneza pracy** (*wynika z miejsca zatrudnienia, sytuacji na świecie, własnych zainteresowań - dlaczego się tym zainteresowałem? dlaczego jest to ciekawy temat? czy jest/będzie zainteresowanie tym tematem*).
   Cel pracy (*cel opracowania, przeznaczenie, potrzeby klienta – dlaczego chce zmian i co chce osiągnąć, podstawowe zadanie, zakres). Wynika to ze schematu zamieszczonego poniżej i przechodzi w opis wizji systemu:*



   Wizja systemu: System <NAZWA> **jest to** <WIĘKSZA KATEGORIA> **w** <CZYJA? GDZIE?> **dla** <GŁÓWNI UŻYTKOWNICY> **odpowiedzialna za** <GŁÓWNE USŁUGI, FUNKCJONALNOŚCI>.
   Określenie miar sukcesu (np. KPI – kluczowe wskaźniki efektywności).

5. **Analiza zadania**:
   a. **Określenie wymagań ogólnych** (*do czego aplikacja będzie wykorzystywana i w jakim celu, jaki jest cel biznesowy, kontekst wykorzystania aplikacji, wymagania ułatwiające życie i pracę odbiorców, można się posłużyć Design Thinking - mapą empatii oraz Matrycą propozycji wartości*).



   b. **Specyfikacja grupy docelowej** (*udziałowcy systemu – biznes + klient + użytkownik. zainteresowane branże, grupy zawodowe lub społeczne, obszary geograficzne, kim będą użytkownicy systemu? jaką posiadają wiedzę? jaki jest ich potencjał w zakresie nauki obsługi aplikacji? – wiek, płeć, wykształcenie, doświadczenie, motywacja, umiejętności – można skorzystać z formularza persony).*

c. **Przegląd i analiza rozwiązań konkurencyjnych** (*żeby nie były to przekopiowane reklamy z Internetu, tylko porównanie funkcji, wady i zalety, wykorzystywane technologie, ceny, wdrożenie, licencjonowanie – najlepiej tabela porównawcza + wnioski*).

d. **Wymagania funkcjonalne** (*podstawowe funkcje systemu wynikające z historii użytkownika «user stories», przypadków użycia «use cases» oraz scenariuszy wynikających z obowiązujących procedur*).

e. **Wymagania niefunkcjonalne** (*niezawodność, czas reakcji, bezpieczeństwo, technologia, kopia danych, współpraca z innymi systemami, reagowanie systemu na błędy użytkownika, ograniczenia, wymagania dotyczące użyteczności i budowy interfejsu*).

f. **Określenie ról użytkowników** (*wyodrębnienie grup użytkowników wykonujących te same zadania i posiadających takie same uprawnienia w systemie*).

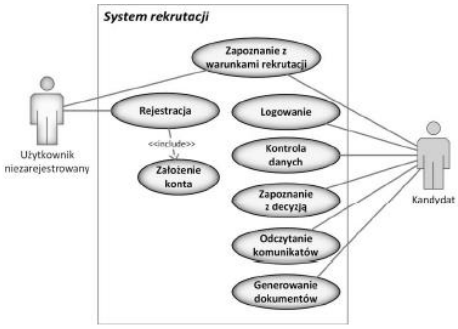
g. **Określenie wymagań** wynikających z przepisów prawa, technologii (*cookies, prawo telekomunikacyjne, RODO, polityka prywatności, prawo autorskie …*).

h. **Wytyczne dotyczące interfejsu** (*układ graficzny, kolorystyka, czcionki, ikony, spełnianie wymagań standardu WCAG …*).

6. **Struktura aplikacji**, czyli statyczna i dynamiczna budowa systemu: warstwy, moduły, komponenty, dane, interfejsy, kluczowe przepływy:

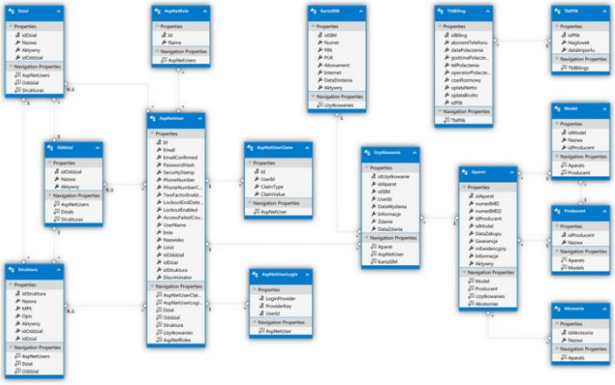
a. **Styl architektoniczny** (np. warstwowa, Clean, Monolit, SOA, Serverless, REST, MVC/MVP/MVVM).

b. **Zadania użytkowników z poszczególnych ról** (*wykaz zadań + diagramy UML np. diagramy przypadków użycia*).

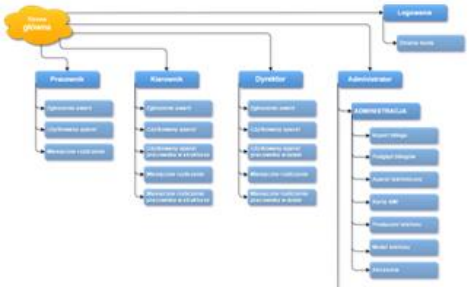


c. **Scenariusze zadań** (w jakiej kolejności obiekty wymieniają komunikaty, aby zrealizować konkretny przypadek użycia – diagram UML sekwencji z uczestnikami w czasie).

d. **Struktura danych** (model danych, obiekty domenowe – klasy i diagram klas, system ORM, opis encji, diagram ERD).



7. **Projekt UI i struktura aplikacji** (zaprojektowanie spójnego i dostępnego interfejsu użytkownika *w* aplikacjach typu *MS Visio, Pencil «lo-fi», Figma «hi-fi»)*
   a. **Struktura informacji i nawigacja** (schemat struktury, typy nawigacji, zasada minimalizacji kroków).
   b. **Układ i hierarchia** (siatka kompozycyjna, hierarchia wizualna).
   c. **Kolorystyka** (paleta podstawowa, kontrast).
   d. **Typografia** (dobór krojów, skala typograficzna, czytelność).
   e. **Grafika** (zdjęcia na odpowiednich licencjach, ikonografia, favicon oraz piktogramy).
   f. **Formularze i komunikaty** (etykiety i pola, walidacja, dostęp z klawiatury).
   g. **Komponenty** (stany elementów, zasady ponownego użycia).
   h. **Responsywność i skalowanie** (punkty przełamania, adaptacja na urządzeniach dotykowych – wielkość pól i odstępy pomiędzy nimi).
   i. **Dostępność** (WCAG 2.2 AA).
   j. **Prototypy** (poszczególne ekrany, powiązania pomiędzy nimi).



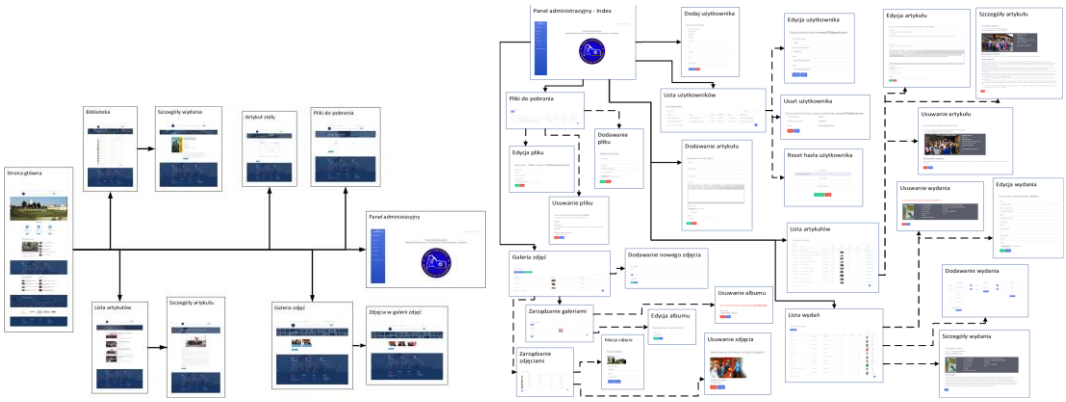
8. **Wykorzystane technologie i narzędzia** (*opis technologii; języka programowania np. Java, C#; opis bibliotek i frameworków np. jQuery, Bootstrap; opis środowiska programistycznego np. VisualStudio, AndroidStudio, opis dodatkowych narzędzi np. ORM – Hibernate, EntityFramework, Room*)
9. **Przygotowanie środowiska pracy** (repozytorium w serwisie GitHub):
   a. **Struktura plików i katalogów** (README.MD, LICENSE, .editorconfig, .gitignore, CHANGELOG.md; src, tests, docs, Dockerfile).
   b. **Model gałęzi** (gałęzie: *main, feat/..., fix/..., chore/..., test/..., docs/, perf/...,* itd.).
   c. **Projekt z tablicą Kanban** (powiązanie zgłoszeń *issues* z elementami tablicy *items*).
   d. **Praca zespołowa** (*fork, clone, push, fetch, merge*).
   e. **Polityka commitów** (każda logiczna zmiana to kolejny *commit*).
   f. **Opisy commitów** (krótki opis z czasownikiem w trzeciej osobie i w czasie teraźniejszym: *„dodaje, poprawia, wprowadza”* - nie tylko *„co”,* ale i *„dlaczego”*; dodatkowe informacje oraz stopka z powiązanym #zgłoszeniem – *Refs* lub *Closes*).
   g. **Pull requesty** (wymuszenie na gałęzi *main*, blokowanie bezpośrednich *commitów* do gałęzi *main*).
   h. **Wersjonowanie** (trzy liczby MAJOR.MINOR.PATCH: wydanie_główne, wersja_poboczna, poprawka, np. 1.1.2).
   i. **Ciągła integracja CI** (workflow w GitHub Action).
10. **Implementacja - praktyczna realizacja zadania** (*opis budowy aplikacji z przykładowymi fragmentami kodu pokazującymi umiejętności programistyczne. Nie wystarczą same zrzuty ekranu!*
   a. Zalecany **opis** funkcji/metody/akcji/widoku, **fragment kodu** prezentujący opisane zagadnienie i dopiero **zrzut ekranu**, jeżeli opis dotyczy interfejsu użytkownika.

opis metody wyświetlającej wiadomości na stronie z kodem (kontroler)

kod odpowiadający za wyświetlanie zdjęć i opisów (widok)

wygląd strony z wyświetlonymi ostatnimi wiadomościami

   b. Każdy fragment kodu, rysunek, tabela musi być **podpisany** i **ponumerowany**.
   c. Należy pamiętać o opisie **walidacji** danych i bezpieczeństwie aplikacji (autoryzacja, ochrona przed atakami, logowanie i diagnostyka).
   d. Poszczególne fragmenty kodu muszą być **testowane**.
   e. Wskazane jest umieszczenie **mapy aplikacji** ukazującej wszystkie ekrany.



11. **Podsumowanie z wnioskami** (*co było celem pracy; co zrobiono, aby go uzyskać, czyli krótki opis aplikacji; o co można aplikację rozbudować, co zmienić lub poprawić – umiejętność krytycznego myślenia i dostrzeganie błędów oraz szans*).
12. **Bibliografia i netografia** (*co najmniej 5 książek + źródła internetowe – wszystkie muszą być wcześniej podane jako źródła w treści pracy i oznaczone przypisem*)
13. **Spis tabel, rysunków i kodu** (*każdy oddzielnie*).
14. **Załączniki** (*oświadczenie o wykorzystaniu AI oraz np. potwierdzenie wdrożenia, umowa-zlecenie na wykonanie, opinie użytkowników, wypełniony przez eksperta formularz Ravdena-Johnsona…*).